

Wengert-Graph Global Optimizer (WGGO)

A Hierarchical ILP Solver for Joint Optimization of Pruning, Sharding, Fusion, Precision, and Memory in the NeuralScript Compiler

Authors: Brandon Wiemer

Date: April 2026

Status: Research Proposal

Abstract

We propose **WGGO (Wengert-Graph Global Optimizer)**, a hierarchical optimization pass at the heart of `ns1-codegen` that jointly solves the decisions made by all six previously proposed compiler-native training techniques — WRGA (adapter placement), CSHA (attention fusion), FASE (gradient accumulation), PCA (packing), CEP (pruning/NAS), and CPDT (sharding/precision/experts) — as a single constrained optimization problem. Without WGGO, each technique runs as an independent compiler pass, making locally-optimal but globally-suboptimal decisions that can conflict: CSHA might fuse operations that CEP wants to prune; CPDT might shard a layer that WRGA wants to keep local for adapter training; FASE might fuse an optimizer step that CPDT wants to distribute. WGGO resolves these conflicts by formulating the combined decision space as a **hierarchical Integer Linear Program** over the model's Wengert graph — NSL's flat DAG of elementary operations — with the cost model as the objective function and hardware constraints as bounds. Using Alpha-inspired hierarchical decomposition, the ILP solves in seconds, not hours, making whole-program global optimization practical as a standard compiler pass.

1. The Interaction Problem

Our six compiler-native techniques make the following decisions:

Technique	Decision Variables	Per-Layer Choices
WRGA	Adapter placement, rank per layer	place/skip $\times$ rank $\in \{0,2,4,8,16\} \rightarrow 5$
CSHA	Fusion level per attention sublayer	$\{\text{none}, \text{level1}, \text{level2}, \text{level3}\} \rightarrow 4$
FASE	Accumulation strategy, fused/separate optimizer	$\{\text{separate}, \text{fused_deferred}\} \rightarrow 2$
PCA	Packing strategy per attention kernel	$\{\text{none}, \text{segment_id}, \text{tile_skip}, \text{multi_seq}\} \rightarrow 4$
CEP	Prune/keep per head, FFN width per layer	heads: 2^H , FFN: k widths $\rightarrow$ large
CPDT	Shard config, optimizer precision, expert placement	ZeRO stages $\times$ precisions $\times$ placements $\rightarrow$ large

Combined decision space for an 8-layer transformer:

- WRGA: $5^8 = 390\text{K}$ (adapter rank per layer)
- CSHA: $4^8 = 65\text{K}$ (fusion level per layer)
- CEP: $\sim 2^{64}$ (per-head pruning) — intractable without decomposition
- CPDT: ~ 1000 (ZeRO configs) $\times 4^8$ (precision per layer) = 65M

Naive enumeration: **$>10^{20}$ configurations**. Even with NSL's 50ms compilation oracle, exhaustive search would take longer than the age of the universe.

1.1 Why Greedy Fails

Running each pass independently produces conflicts:

Conflict 1: CSHA vs CEP. CSHA Level 2 fuses Q/K/V projection into the attention kernel. But CEP may want to prune 2 of 8 heads, which changes the projection dimensions. If CSHA fuses first, the fused kernel has fixed dimensions that can't be pruned. If CEP prunes first, CSHA must generate a fused kernel for the pruned dimensions — which may have different SMEM requirements and may no longer be fusible.

Conflict 2: WRGA vs CPDT. WRGA places LoRA adapters on layers with high roofline slack. CPDT shards those same layers across GPUs. The adapter's backward gradient must be all-reduced, but WRGA's dead-gradient-elimination assumes local computation. If WRGA runs before CPDT, its pruned backward doesn't account for communication. If CPDT runs before WRGA, its sharding plan doesn't account for adapter locality.

Conflict 3: FASE vs CPDT. FASE fuses the optimizer step into the backward pass per-layer. CPDT uses reduce-scatter to distribute gradients across GPUs. FASE's fused step must happen AFTER the reduce-scatter completes — but FASE wants to fuse the step DURING the backward. The two optimizations need to be co-scheduled: the fused step runs on each GPU's gradient shard after reduce-scatter, not on the full gradient.

Conflict 4: CEP vs CPDT. CEP prunes different numbers of heads per layer, creating heterogeneous layer dimensions. CPDT's ZeRO sharding assumes homogeneous layers for efficient bucketing. Non-uniform layers require per-layer shard sizes, which increases communication overhead.

Resolution requires a global view. No ordering of independent passes resolves all four conflicts. The decisions must be made jointly.

2. The Wengert-Graph Formulation

2.1 The Wengert Graph as Optimization Input

NSL's source AD (M40) extracts the model's forward computation into a **Wengert list** — a flat sequence of elementary operations with explicit data dependencies. WGGO lifts this into a **Wengert graph** $W = (V, E)$:

- **Vertices V:** Each elementary operation (matmul, add, softmax, rmsnorm, etc.)
- **Edges E:** Data dependencies (operation i produces a tensor consumed by operation j)
- **Vertex annotations:** op type, tensor shapes, estimated FLOP/byte (from cost model)
- **Edge annotations:** tensor size, dtype, device

The Wengert graph is the natural representation for joint optimization because ALL six techniques operate on it:

Technique	Wengert Graph Operation
WRGA	Insert adapter vertices between projection vertices
CSHA	Merge adjacent vertices into fused super-vertices
FASE	Add optimizer-step vertices after gradient vertices
PCA	Annotate attention vertices with segment-ID config
CEP	Remove vertices (pruned heads/neurons)
CPDT	Partition vertices across device meshes

2.2 Decision Variables

For each vertex $v \in V$ (or vertex group for structured decisions), WGGO defines integer decision variables:

```
For each layer  $l \in \{0, \dots, L-1\}$ :

# CEP decisions (pruning)
 $h[l, k] \in \{0, 1\}$            # keep head  $k$  in layer  $l$ ? (binary)
 $f[l] \in \{f_1, f_2, \dots, f_m\}$  # FFN width for layer  $l$  (discrete set)

# CSHA decisions (fusion)
 $c[l] \in \{0, 1, 2, 3\}$        # CSHA fusion level for layer  $l$ 

# WRGA decisions (adapters)
 $r[l, w] \in \{0, 2, 4, 8, 16\}$  # adapter rank for weight matrix  $w$  in layer  $l$ 

# CPDT decisions (sharding + precision)
 $s_p[l] \in \{1, \dots, N\}$      # parameter shard factor for layer  $l$ 
 $s_g[l] \in \{1, \dots, N\}$      # gradient shard factor for layer  $l$ 
 $p_m[l] \in \{8, 16, 32\}$       # optimizer  $m$  precision (bits)
 $p_v[l] \in \{8, 16, 32\}$       # optimizer  $v$  precision (bits)

# FASE decisions (accumulation)
 $a[l] \in \{0, 1\}$              # fuse optimizer into backward for layer  $l$ ?
```

2.3 Objective Function

```
minimize: total_step_time(decisions)
=  $\sum_l$  [ forward_time( $l$ ,  $h[l]$ ,  $f[l]$ ,  $c[l]$ )
    + backward_time( $l$ ,  $h[l]$ ,  $f[l]$ ,  $r[l]$ ,  $a[l]$ )
    + comm_time( $l$ ,  $s_p[l]$ ,  $s_g[l]$ )
    + optimizer_time( $l$ ,  $p_m[l]$ ,  $p_v[l]$ ,  $a[l]$ ) ]
```

Each term is computed by NSL's cost model — the key enabler. The cost model evaluates any configuration of a layer in ~1ms (single layer analysis), making the objective function cheap to evaluate.

2.4 Constraints

```

# Memory constraint (per GPU)
 $\sum_l [ \text{param\_mem}(l, h[l], f[l]) / s_p[l]$ 
    +  $\text{optim\_mem}(l, h[l], f[l], p_m[l], p_v[l]) / s_{os}[l]$ 
    +  $\text{activation\_mem}(l, h[l], f[l], c[l]) ] \leq \text{GPU\_memory}$ 

# Shape compatibility (from shape algebra)
 $\forall l: \text{output\_dim}(l, h[l], f[l]) == \text{input\_dim}(l+1, h[l+1], f[l+1])$ 
    OR  $\text{reshape\_possible}(\text{output\_dim}(l), \text{input\_dim}(l+1))$ 

# CSHA fusion feasibility (from SMEM analysis)
 $\forall l: c[l] \geq 2 \rightarrow \text{smem\_required}(l, h[l], f[l]) \leq \text{GPU\_smem}$ 

# WRGA adapter compatibility with CPDT sharding
 $\forall l: r[l,w] > 0 \rightarrow \text{adapter\_comm}(l, r[l,w], s_g[l]) \leq \text{budget}$ 

# CEP pruning minimum quality (from weight importance)
 $\sum_{l,k} h[l,k] \times \text{importance}(l,k) \geq \text{min\_retained\_importance}$ 

# FASE compatibility with CPDT
 $\forall l: a[l] = 1 \text{ AND } s_g[l] > 1 \rightarrow \text{fused\_step\_after\_reduce}(l) = 1$ 

# Numerical stability for low-precision optimizer
 $\forall l: \text{sensitivity}(l) > \text{threshold} \rightarrow p_m[l] \geq 16 \text{ AND } p_v[l] \geq 16$ 

```

3. Hierarchical Decomposition

The full ILP over all variables for all layers is tractable for small models but can be slow for large ones. WGGO uses **Alpa-inspired hierarchical decomposition** — but extended to cover all six techniques, not just parallelism.

3.1 Three-Level Hierarchy

Level 1: Inter-Layer Decisions (coarse)

Variables: per-layer pruning (which layers to keep/thin),
pipeline stage assignment, ZeRO sharding strategy

Method: Dynamic Programming (DP) over layer graph

Time: $O(L^2 \times S)$ where L =layers, S =strategy candidates per layer

Level 2: Intra-Layer Decisions (medium)

Variables: per-head pruning, FFN width, fusion level,
adapter rank, optimizer precision

Method: ILP per layer (independent once inter-layer decisions fix layer shapes)

Time: $O(L \times \text{ILP_solve_per_layer})$ – ILP per layer has ~20-50 variables

Level 3: Intra-Kernel Decisions (fine)

Variables: tile sizes, register allocation, SMEM layout,
communication schedule within fused kernels

Method: Cost-model-guided heuristic (existing M31/M37 infrastructure)

Time: $O(L \times K)$ where K =kernels per layer

3.2 Level 1: Inter-Layer DP

The inter-layer pass determines the coarse structure:

State: (layer_index, pipeline_stage, accumulated_memory, accumulated_params)

Transitions: for each layer l , enumerate:

- Keep layer l (full width, thinned width, or pruned to specific head count)
- Assign to pipeline stage s
- Choose ZeRO shard factor for this stage

DP recurrence:

$\text{cost}(l, s, \text{mem}) = \min \text{ over all } (\text{layer_config}, \text{stage_assignment}):$
 $\text{cost}(l-1, s', \text{mem}') + \text{layer_time}(l, \text{layer_config}, \text{stage_assignment})$
subject to $\text{mem}' + \text{layer_mem}(l, \text{layer_config}) \leq \text{GPU_memory}$

Time complexity: $O(L \times S \times M)$ where $L \approx 8-96$ layers, $S \approx 4-8$ stages, $M \approx 10-20$ memory buckets
= ~10K-100K states. Solved in milliseconds.

Output: Per-layer coarse configuration (keep/prune, pipeline stage, shard group).

3.3 Level 2: Intra-Layer ILP

For each layer, given the inter-layer decisions, solve a small ILP:

```
Variables per layer l:
  h[k] ∈ {0,1} for k=1..H           # head pruning (H ≤ 32)
  f ∈ {f1, ..., fm}               # FFN width (m ≤ 5)
  c ∈ {0, 1, 2, 3}                   # CSHA level
  r[w] ∈ {0, 2, 4, 8, 16} for w=1..W # adapter ranks (W ≤ 6)
  p_m ∈ {8, 16, 32}                  # optimizer m precision
  p_v ∈ {8, 16, 32}                  # optimizer v precision

Total variables: H + 1 + 1 + W + 2 ≈ 20-50 per layer
Constraints: ~15-30 per layer
```

This is a tiny ILP — solvable in <100ms per layer by any standard solver (branch-and-bound, even brute force for $H \leq 8$).

Key: the ILP's objective function calls the cost model. Each candidate configuration is evaluated by `cost_model.layer_time(l, config)` which returns the exact estimated time in microseconds. This is not a surrogate or proxy — it's the actual roofline-bounded latency estimate from M37.

3.4 Level 3: Intra-Kernel Heuristics

Given the ILP's decisions (which heads exist, what fusion level, what adapter ranks), the existing compiler passes generate optimized kernels:

- M31 (fusion) generates fused PTX for the selected CSHA level
- M37 (cost model) selects optimal tile sizes
- M36 (memory planner) computes activation schedule
- `flash_attention.rs` generates the attention kernel variant

These are fast heuristic passes that already exist in NSL. WGGO simply feeds them the globally-optimal configuration instead of a locally-determined one.

4. The WGGO Solver Architecture

Input: Model AST + optional weights + cluster spec + training config

WGGO Compiler Pass

1. Wengert Graph Extraction (M40)
AST → flat DAG of elementary ops
Time: ~10ms
 2. Cost Model Annotation (M37)
Per-vertex FLOP, bytes, AI, bound
Time: ~20ms
 3. Weight Analysis (M52) [if --weights]
Per-head importance, spectral energy
Time: ~200ms
 4. Level 1: Inter-Layer DP
Pipeline stages, ZeRO config, pruning
Time: ~50ms
 5. Level 2: Per-Layer ILP
Heads, FFN, fusion, adapters, precision
Time: ~100ms per layer × L layers
(parallelizable across CPU cores)
 6. Level 3: Kernel Generation
Fused PTX, tile sizes, SMEM layout
Time: ~50ms per layer
 7. Memory Planning (M36)
Global activation schedule
Time: ~100ms
 8. Communication Schedule (CPDT)
Allgather/reducescatter placement
Time: ~50ms
- Total compilation time:
- 8-layer model: ~1.5 seconds
 - 32-layer model: ~5 seconds
 - 96-layer model: ~15 seconds

Output: Globally-optimized Cranelift IR with:

- Pruned model topology (CEP decisions)

- Fused attention kernels (CSHA decision)
- Adapter-augmented layers (WRGA decisions)
- Quantized optimizer steps (FASE + CPDT precision)
- Communication schedule (CPDT sharding)
- Packing-aware kernels (PCA)
- Memory plan (all techniques composed)

4.1 Why This Is Fast

The three-level hierarchy is the key to tractability:

1. **Level 1 (DP)** reduces the combinatorial explosion by fixing coarse structure first. With $L=32$ layers and ~ 10 coarse options per layer, the DP explores ~ 320 states, not 10^{32} .
2. **Level 2 (ILP)** solves *per-layer* with ~ 30 variables each. A 30-variable binary ILP has $\sim 10^9$ possible assignments, but branch-and-bound with cost-model-guided bounds typically solves in $< 100\text{ms}$. Furthermore, the L per-layer ILPs are **independent** (once inter-layer decisions are fixed) and can be solved in parallel.
3. **Level 3 (heuristic)** is the existing compiler infrastructure — fast by design.

Compare to Alpa: Alpa uses ILP for intra-operator parallelism only (one concern) and takes 30-60 seconds for compilation. WGGO solves six concerns jointly in comparable time because:

- NSL's cost model is built-in (no external profiling needed)
- The Wengert graph is available natively (no JAX tracing overhead)
- The hierarchical decomposition exploits transformer structure (repeated layers)

4.2 Exploiting Transformer Repetition

Most transformers have repeated identical layers. WGGO exploits this:

For a model with L identical layers:

- Level 1 DP: solve once for the "template" layer, then specialize per-layer
- Level 2 ILP: solve once for the template, then allow per-layer deviations only where weight analysis shows significant differences

For NSLCoder-50M (8 identical blocks), WGGO solves the Level 2 ILP **once** and then runs 8 fast constraint-checks to verify the solution works for each layer's specific weights. This reduces Level 2 time from $8 \times 100\text{ms} = 800\text{ms}$ to $\sim 150\text{ms}$.

5. The ILP Formulation (Level 2 Detail)

For a single transformer layer l with H attention heads, the ILP is:

```

minimize:
    forward_time(h, f, c) + backward_time(h, f, r, a) + comm_time(s_g) +
    optim_time(p_m, p_v, a)

subject to:
    # Memory
    param_bytes(h, f) / s_p + optim_bytes(h, f, p_m, p_v) / s_os + act_bytes(h, f,
c) ≤ M_budget

    # Pruning quality
     $\sum_k h[k] \times \text{importance}[k] \geq \text{min\_importance}$ 

    # At least 1 head
     $\sum_k h[k] \geq 1$ 

    # GQA compatibility: kept heads must include complete KV groups
     $\forall \text{ group } g: (\sum_{k \in g} h[k] = 0) \text{ OR } (\sum_{k \in g} h[k] = |g|)$ 

    # Fusion feasibility
     $c \geq 2 \rightarrow \text{smem}(h, f) \leq \text{SMEM\_max}$ 
     $c = 3 \rightarrow \text{smem}(h, f) + \text{ffn\_smem}(f) \leq \text{SMEM\_max} \quad \# \text{ full block fusion}$ 

    # Adapter-shard compatibility
     $r[w] > 0 \text{ AND } s_g > 1 \rightarrow \text{adapter\_comm\_cost} \leq \text{threshold}$ 

    # FASE-shard compatibility
     $a = 1 \text{ AND } s_g > 1 \rightarrow \text{fused\_after\_reduce} = 1$ 

    # Numerical stability
     $\text{sensitivity} \geq \text{high\_threshold} \rightarrow p_m \geq 16$ 
     $\text{sensitivity} \geq \text{critical\_threshold} \rightarrow p_m \geq 32 \text{ AND } p_v \geq 32$ 

```

Linearization: The objective function $\text{forward_time}(h, f, c)$ is nonlinear in the decision variables. WGGO linearizes it using a **piecewise-linear approximation**: pre-compute the cost model's output for each combination of (h_count, f, c) and store as lookup tables. The ILP then selects among these pre-computed costs.

For $H=8$ heads, the number of distinct head-count values is 8. With 5 FFN widths and 4 fusion levels, the lookup table has $8 \times 5 \times 4 = 160$ entries per layer — trivially small.

6. Handling the ILP Solver

6.1 Built-In Solver

WGGO includes a lightweight branch-and-bound ILP solver written in Rust, embedded in `ns1-codegen`. This avoids external dependencies (no CPLEX, Gurobi, or GLPK required).

The solver is specialized for WGGO's problem structure:

- **Small variable count** (20-50 per layer): exhaustive search with branch-and-bound is fast
- **Pre-computed objective values**: the cost model lookup tables eliminate function calls during search
- **Strong bounds from cost model**: the relaxed LP lower bound is tight because the cost model is nearly linear in most regions
- **Warm starting from greedy**: the greedy solution (from running each technique independently) provides an excellent initial feasible solution that the ILP improves upon

6.2 Fallback: Greedy with Conflict Resolution

For users who want faster compilation at the cost of optimality, WGGO offers a greedy mode:

WGGO-Greedy Algorithm:

1. Run each technique independently (existing passes)
2. Detect conflicts between decisions
3. Resolve conflicts using priority ordering:
CEP > CPDT > CSHA > WRGA > FASE > PCA
(structural decisions override optimization decisions)
4. Re-evaluate resolved configuration with cost model
5. If cost increased by >5%: run Level 2 ILP on conflicting layers only

This greedy-with-targeted-ILP approach handles 90% of cases in ~500ms (vs ~2s for full WGGO).

7. NSL Language Surface

7.1 Automatic (Default)

ns!

```
# WGG0 runs automatically when multiple techniques are active
@wrga(mode=auto, budget=100K)
@cep_prune(sparsity=0.3)
@cpdt(cluster={gpus=8, intra_bw="900GB/s"})
train(model=m, epochs=3, grad_clip=1.0):
    optimizer: AdamW(lr=1e-4)
    step(batch):
        let logits = m.forward_train(batch.input_ids, true)
        let loss = cross_entropy(logits, batch.labels)
```

7.2 CLI

```
bash
```

```
$ nsl build --wggo --weights model.safetensors --target 8xH100 model.nsl
```

```
=== WGGO Global Optimization Report ===
```

```
Optimization time: 2.3 seconds (Level 1: 0.05s, Level 2: 1.8s, Level 3: 0.45s)
```

```
Level 1 (Inter-Layer):
```

```
  Pipeline: 1 stage (model fits in single mesh)
```

```
  ZeRO: s_p=1, s_g=8, s_os=8
```

```
  Pruning: layers 2-5 thinned, layers 0,1,6,7 full width
```

```
Level 2 (Intra-Layer):
```

```
  Layer 0: 8/8 heads, FFN=1408, CSHA-L2, no adapters, FP16 m/FP32 v
```

```
  Layer 1: 8/8 heads, FFN=1408, CSHA-L2, no adapters, FP16 m/FP32 v
```

```
  Layer 2: 6/8 heads, FFN=1024, CSHA-L1, LoRA r=4 on wq, INT8 m/FP16 v
```

```
  Layer 3: 6/8 heads, FFN=1024, CSHA-L1, LoRA r=4 on wq, INT8 m/FP16 v
```

```
  Layer 4: 6/8 heads, FFN=768, CSHA-L1, LoRA r=8 on wq+wk, INT8 m/INT8 v
```

```
  Layer 5: 6/8 heads, FFN=768, CSHA-L1, LoRA r=8 on wq+wk, INT8 m/INT8 v
```

```
  Layer 6: 7/8 heads, FFN=1024, CSHA-L2, LoRA r=4 on wq, FP16 m/FP32 v
```

```
  Layer 7: 8/8 heads, FFN=1408, CSHA-L2, no adapters, FP32 m/FP32 v
```

```
Conflicts resolved: 3
```

```
[1] CSHA-L2 on layer 4 → downgraded to L1 (pruned heads changed SMEM layout)
```

```
[2] WRGA adapter on layer 0 → removed (CPDT shard makes adapter comm > threshold)
```

```
[3] FASE fused step on layers 2-5 → deferred to after reduce-scatter
```

```
Global metrics:
```

```
  Step time: 3.8ms (vs 4.2ms greedy, vs 5.8ms no optimization)
```

```
  Peak memory: 1.6 GB/GPU (vs 2.0 GB greedy, vs 5.2 GB no optimization)
```

```
  Trainable params: 0.3M adapters + 31.2M pruned model = 31.5M
```

```
  Binary size: 98 MB (vs 195 MB unpruned)
```

```
$ nsl build --wggo=greedy # fast mode: ~500ms, within 5% of optimal
```

8. Why This Is Uniquely NSL

8.1 Alpa Comparison

Alpa pioneered ILP-based intra-operator parallelism search. But:

Capability	Alpa	WGGO
Optimization concerns	1 (parallelism)	6 (pruning, fusion, adapters, accum, packing, sharding)
Cost model	External profiler	Built-in analytical (M37)
Computation graph	Traced at runtime (JaxPR → XLA HLO)	Extracted at compile time (Wengert list)
Kernel generation	Delegates to XLA	Generates PTX directly
Weight-aware decisions	No	Yes (spectral analysis, importance scores)
Solver input	Parallelism strategies only	Full training configuration
ILP size	Tens of thousands of ops	Decomposed: ~30 vars per layer
Solve time	30-60 seconds	2-5 seconds

8.2 Why PyTorch/DeepSpeed Cannot Build This

1. **No unified cost model.** Each concern (fusion, sharding, optimizer precision) has its own profiling methodology. There's no single objective function that evaluates a joint configuration.
2. **No Wengert graph.** PyTorch's autograd graph is built dynamically during forward execution. It cannot be analyzed statically before training.
3. **No kernel generation control.** PyTorch delegates to pre-compiled kernels (cuBLAS, FlashAttention, cuDNN). The solver cannot change kernel configurations because it doesn't generate them.
4. **No weight-aware analysis.** Pruning importance, spectral analysis, and sensitivity scoring require weight access at "compile time" — a concept that doesn't exist in PyTorch's workflow.
5. **Separate tools, separate passes.** DeepSpeed ZeRO, bitsandbytes, LLM-Pruner, LoRA/PEFT, and FlashAttention are independent libraries with no shared optimization surface. DeepCompile is moving toward unification, but is limited by PyTorch's operator-level graph representation.

9. Implementation Plan

Component	Location	LOC (est.)	Description
Wengert graph builder	<code>ns1-codegen/src/wggo_graph.rs</code>	400	Build optimization graph from Wengert list
Cost model interface	<code>ns1-codegen/src/wggo_cost.rs</code>	300	Wrap M37 cost model for ILP objective evaluation
Level 1 DP solver	<code>ns1-codegen/src/wggo_dp.rs</code>	500	Inter-layer dynamic programming
Level 2 ILP solver	<code>ns1-codegen/src/wggo_ilp.rs</code>	800	Branch-and-bound ILP with cost-model LUTs
Conflict detector	<code>ns1-codegen/src/wggo_conflicts.rs</code>	300	Identify inter-technique conflicts
Decision applicator	<code>ns1-codegen/src/wggo_apply.rs</code>	400	Apply ILP solution to AST/codegen
WGGO orchestrator	<code>ns1-codegen/src/wggo.rs</code>	300	Main pass: coordinate levels 1-3

Total: ~3,000 LOC. The solver is compact because it delegates heavy lifting to existing infrastructure (cost model, memory planner, kernel generators).

Dependencies: `wengert.rs`, `cost_model.rs`, `memory_planner.rs`, `weight_aware.rs`, `flash_attention.rs`, `epilogue_fusion.rs` — all existing.

10. Conclusion

WGGO is the keystone that makes the six-paper suite practical. Without it, each technique is a standalone compiler pass that produces locally-optimal decisions. With it, the compiler solves a global optimization problem that jointly determines what to prune, where to fuse, how to shard, what precision to use, where to place adapters, and how to schedule communication — all in a single 2-5 second compilation pass.

The key insight: **NSL's cost model makes the objective function cheap.** Classical ILP in compilers is limited by the cost of evaluating each candidate (often requires simulation or profiling). NSL's analytical cost model evaluates any configuration in microseconds, making the ILP's branch-and-bound search fast enough for practical whole-program optimization.

WGGO completes the compiler-native training optimization architecture:

Paper	Role
WRGA	Adapter technique
CSHA	Attention technique
FASE	Gradient accumulation technique
PCA	Sequence packing technique
CEP	Pruning/NAS technique
CPDT	Distributed training technique
WGGO	Global solver that composes all six

The compiled binary is the optimal training plan — globally optimized across all six concerns, with no runtime configuration, no trial-and-error, and no conflicting pass decisions.

References

1. Zheng, L., et al. "Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning." OSDI 2022.
2. Jia, Z., et al. "Beyond Data and Model Parallelism for Deep Neural Networks (FlexFlow)." MLSys 2019.
3. Shobaki, G., et al. "Combinatorial Register Allocation and Instruction Scheduling." ACM TACO 2013.
4. Chang, C., et al. "Using Integer Linear Programming for Instruction Scheduling and Register Allocation." Computers & Mathematics 1997.
5. Unger, C., et al. "Unity: Accelerating DNN Training Through Joint Optimization of Algebraic Transformations and Parallelization." OSDI 2022.
6. DeepSpeed Team. "DeepCompile: Compiler-Level Optimizations for Distributed Training." 2025.